

OS Mod-3: Inter process Communication

Concurrency in Operating Systems:

Concurrency is the execution of multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel. The running process threads always communicate with each other through shared memory or message passing. Concurrency results in the sharing of resources resulting in problems like deadlocks and resource starvation.

It helps in techniques like coordinating the execution of processes, memory allocation, and execution scheduling for maximizing throughput.

There are several motivations for allowing concurrent execution

- Physical resource Sharing: Multiuser environment since hardware resources are limited
- Logical resource Sharing: Shared file (same piece of information)
- Computation Speedup: Parallel execution
- Modularity: Divide system functions into separation processes

Issues of Concurrency:

1. Non-atomic: Operations that are non-atomic but interruptible by several processes may cause issues. A non-atomic operation depends on other processes, and an atomic operation runs independently of other processes.
2. Deadlock: In concurrent computing, it occurs when one group member waits for another member, including itself, to send a message and release a lock. Software and hardware locks are commonly used to arbitrate shared resources and implement process synchronization in parallel computing, distributed systems, and multiprocessing.
3. Blocking: A blocked process is waiting for some event, like the availability of a resource or completing an I/O operation. Processes may block waiting for resources, and a process may be blocked for a long time waiting for terminal input. If the process is needed to update some data periodically, it will be very undesirable.
4. Race Conditions: A race problem occurs when the output of a software application is determined by the timing or sequencing of other uncontrollable events. Race situations can also happen in multithreaded software, run in a distributed environment, or are interdependent on shared resources.
5. Starvation: A problem in concurrent computing is where a process is continuously denied the resources it needs to complete its work. It could be caused by errors in scheduling or mutual exclusion algorithms, but resource leaks may also cause it.

Concurrent system design frequently requires developing dependable strategies for coordinating their execution, data interchange, memory allocation, and execution schedule to decrease response time and maximize throughput.

Advantages

1. **Better Performance:** It improves the operating system's performance. When one application only utilizes the processor, and another only uses the disk drive, the time it takes to perform both apps simultaneously is less than the time it takes to run them sequentially.
2. **Better Resource Utilization:** It enables resources that are not being used by one application to be used by another.
3. **Running Multiple Applications:** It enables you to execute multiple applications simultaneously.

Disadvantages

1. It is necessary to protect multiple applications from each other.
2. It is necessary to use extra techniques to coordinate several applications.
3. Additional performance overheads and complexities in OS are needed for switching between applications.

Principles of Concurrency: Both interleaved and overlapped processes can be viewed as examples of concurrent processes, they both present the same problems. The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes
- The way operating system handles interrupts
- The scheduling policies of the operating system

Critical Section Problem in OS (Operating System)

Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device. The critical section cannot be executed by more than one process at the same time; the operating system faces the difficulties in allowing and disallowing the processes from entering the critical section. The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise. In order to synchronize the cooperative processes, our main task is to solve the critical section problem. We need to provide a solution in such a way that the following conditions can be satisfied.

Requirements of Synchronization mechanisms

- **Primary:**
 - **Mutual Exclusion:** Our solution must provide mutual exclusion. By Mutual Exclusion, we mean that if one process is executing inside the critical section then the other process must not enter in the critical section.

- Progress: Progress means that if one process doesn't need to execute into the critical section then it should not stop other processes from getting into the critical section.
- Secondary:
 - Bounded Waiting: We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.
 - Architectural Neutrality: Our mechanism must be architecturally natural. It means that if our solution is working fine on one architecture then it should also run on the other ones as well.

Pipes:

Pipe is a communication medium between two or more related or interrelated processes. It can be either within one process or a communication between the child and the parent processes. Communication can also be multi-level such as communication between the parent, the child and the grand-child, etc. Communication is achieved by one process writing into the pipe and another reading from the pipe. To achieve the pipe system call, create two files, one to write into the file and another to read from the file. This system call would create a pipe for one-way communication i.e., it creates two descriptors, first one is connected to read from the pipe and other one is connected to write into the pipe.

In operating systems, a pipe is a method of inter-process communication (IPC) that allows data to flow from one process to another. Pipes are used to enable processes to communicate with each other and synchronize their actions by sending data in a unidirectional manner. There are primarily two types of pipes:

1. Anonymous Pipes:

Description: These are the simplest type of pipes and are used for communication between related processes, typically between a parent and a child process. They are called "anonymous" because they do not have a name and are only used while the processes are still running.

Characteristics:

- Unidirectional: Data flows in one direction only, from the write end to the read end.
- Scope: Usually limited to the parent-child relationship between processes.
- Usage: Often used in command-line operations and scripting to pass the output of one command as input to another (e.g., `ls | grep "file"` in Unix-like systems).

Implementation: In Unix-like systems, anonymous pipes are created using the `pipe()` system call, which returns a pair of file descriptors: one for reading and one for writing.

2. Named Pipes (FIFOs):

Description: Named pipes, also known as FIFOs (First In, First Out), are similar to anonymous pipes but with a key difference: they have a name and can be accessed by unrelated processes.

They are more flexible and persistent because their existence is not tied to the lifespan of any particular process.

Characteristics:

- Bidirectional: Though traditionally unidirectional, named pipes can be used bidirectionally by creating separate pipes for reading and writing.
- Scope: Can be used by any processes that have access to the named pipe, regardless of their parent-child relationship.
- Persistence: The named pipe persists in the filesystem until explicitly removed, allowing processes to communicate across different executions.

Implementation: In Unix-like systems, named pipes are created using the `mkfifo()` system call, which creates a special file in the filesystem that acts as a pipe. Processes can then open this file for reading or writing.

Mutual Exclusion in Synchronization:

During concurrent execution of processes, processes need to enter the critical section (or the section of the program shared across processes) at times for execution. It might happen that because of the execution of multiple processes at once, the values stored in the critical section become inconsistent. In other words, the values depend on the sequence of execution of instructions – also known as a race condition. The primary task of process synchronization is to get rid of race conditions while executing the critical section.

What is Mutual Exclusion?

Mutual Exclusion is a property of process synchronization that states that “no two processes can exist in the critical section at any given point of time“. The term was first coined by Dijkstra. Any process synchronization technique being used must satisfy the property of mutual exclusion, without which it would not be possible to get rid of a race condition.

The need for mutual exclusion comes with concurrency. There are several kinds of concurrent execution:

- Interrupt handlers
- Interleaved, preemptively scheduled processes/threads
- Multiprocessor clusters, with shared memory
- Distributed systems

Mutual exclusion methods are used in concurrent programming to avoid the simultaneous use of a common resource, such as a global variable, by pieces of computer code called critical sections.

The requirement of mutual exclusion is that when process P1 is accessing a shared resource R1, another process should not be able to access resource R1 until process P1 has finished its operation with resource R1.

Examples of such resources include files, I/O devices such as printers, and shared data structures.

Conditions Required for Mutual Exclusion

According to the following four criteria, mutual exclusion is applicable:

- When using shared resources, it is important to ensure mutual exclusion between various processes. There cannot be two processes running simultaneously in either of their critical sections.
- It is not advisable to make assumptions about the relative speeds of the unstable processes.
- For access to the critical section, a process that is outside of it must not obstruct another process.
- Its critical section must be accessible by multiple processes in a finite amount of time; multiple processes should never be kept waiting in an infinite loop.

Approaches To Implementing Mutual Exclusion

- Software Method: Leave the responsibility to the processes themselves. These methods are usually highly error-prone and carry high overheads.
- Hardware Method: Special-purpose machine instructions are used for accessing shared resources. This method is faster but cannot provide a complete solution. Hardware solutions cannot guarantee the absence of deadlock and starvation.
- Programming Language Method: Provide support through the operating system or through the programming language.

Requirements of Mutual Exclusion

- At any time, only one process is allowed to enter its critical section.
- The solution is implemented purely in software on a machine.
- A process remains inside its critical section for a bounded time only.
- No assumption can be made about the relative speeds of asynchronous concurrent processes.
- A process cannot prevent any other process from entering into a critical section.
- A process must not be indefinitely postponed from entering its critical section.

What is a Need of Mutual Exclusion?

An easy way to visualize the significance of mutual exclusion is to imagine a linked list of several items, with the fourth and fifth items needing to be removed. By changing the previous node's next reference to point to the succeeding node, the node that lies between the other two nodes is deleted.

To put it simply, whenever node "i" wants to be removed, node "with - 1"'s subsequent reference is changed to point to node "ith + 1" at that time. Two distinct nodes can be removed by two threads at the same time when a shared linked list is being used by many threads. This occurs when the first thread modifies node "ith - 1" next reference, pointing towards the node "ith + 1," and the second thread modifies node "ith" next reference, pointing towards the node

“ith + 2.” Although both nodes have been removed, the linked list’s required state has not yet been reached because node “i + 1” still exists in the list because node “ith – 1” next reference still points to it.

Now, this situation is called a race condition. Race conditions can be prevented by mutual exclusion so that updates at the same time cannot happen to the very bit about the list.

Synchronization Techniques Include

- Mutex(Mutual Exclusion): only one process can access the critical section at a time, it is a locking mechanism.
- Semaphores: Signaling mechanisms to control access to shared resources.
- Monitors: High-level synchronization constructs that encapsulate shared resources and provide a safe interface for processes to access them.
- Locks : Mechanisms to protect shared resources from concurrent access. Locks can be simple or complex.

Semaphores: Semaphores are just normal variables used to coordinate the activities of multiple processes in a computer system. They are used to enforce mutual exclusion, avoid race conditions, and implement synchronization between processes.

Types of Semaphores: Semaphores are of two Types:

- Binary Semaphore: This is also known as a mutex lock. It can have only two values – 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problems with multiple processes.
- Counting Semaphore: Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Advantages of Semaphores

- A simple and effective mechanism for process synchronization
- Supports coordination between multiple processes
- Provides a flexible and robust way to manage shared resources.
- It can be used to implement critical sections in a program.
- It can be used to avoid race conditions.

Disadvantages of Semaphores

- It Can lead to performance degradation due to overhead associated with wait and signal operations.
- Can result in deadlock if used incorrectly.
- It was proposed by Dijkstra in 1965 which is a very significant technique to manage concurrent processes by using a simple integer value, which is known as a semaphore. A

semaphore is simply an integer variable that is shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.

- It can cause performance issues in a program if not used properly.
- It can be difficult to debug and maintain.
- It can be prone to race conditions and other synchronization problems if not used correctly.
- It can be vulnerable to certain types of attacks, such as denial of service attacks.

Reader-Writer Problem: In Reader-Writer problem Synchronizing access to shared data where multiple readers can read the data simultaneously, but writers need exclusive access to modify it. In simple terms imagine a library where multiple readers and writers come and all readers want to read a book, and some people(writers) want to update or edit the book. That's why we need a system to ensure that these actions are done smoothly without errors or conflicts.

- Multiple Readers and Writers: Processes that need to access a shared resource(data).
- Data: Shared resources represented by a Semaphore.
- Semaphore Value for Reader: Process that reads shared data, Can access simultaneously(value>1, more than one reader can access at the same time)
- Semaphore Value for Writers: Processes that modify shared data need exclusive access(value =1, one at a time)

Solution

Readers: Many readers can read the book at the same time without any issue.

- Acquire the reader count semaphore first.
- If the first reader acquires semaphore(the first reader checks if any writer is editing and LOCKS the book for reading).
- Release the reader count semaphore.
- Read the data.
- Acquire the reader count Semaphore.
- If the last reader, release the writer semaphore(when the last reader finishes reading they UNLOCK the book so a writer can edit if needed).
- Release the reader count semaphore.

Writers: Only one can make the changes to the book at a time.

- Acquire the writer's semaphore(first wait to edit until no readers are reading the book and then lock the book if no reader is reading).
- Write the data.
- Release the writer semaphore(after finishing they UNLOCK the book for others to do the task).

Monitors

Monitors are a higher-level synchronization construct that simplifies process synchronization by providing a high-level abstraction for data access and synchronization. Monitors are implemented as programming language constructs, typically in object-oriented languages, and provide mutual exclusion, condition variables, and data encapsulation in a single construct.

- A monitor is essentially a module that encapsulates a shared resource and provides access to that resource through a set of procedures. The procedures provided by a monitor ensure that only one process can access the shared resource at any given time, and that processes waiting for the resource are suspended until it becomes available.
- Monitors are used to simplify the implementation of concurrent programs by providing a higher-level abstraction that hides the details of synchronization. Monitors provide a structured way of sharing data and synchronization information, and eliminate the need for complex synchronization primitives such as semaphores and locks.
- The key advantage of using monitors for process synchronization is that they provide a simple, high-level abstraction that can be used to implement complex concurrent systems. Monitors also ensure that synchronization is encapsulated within the module, making it easier to reason about the correctness of the system.

However, monitors have some limitations. For example, they can be less efficient than lower-level synchronization primitives such as semaphores and locks, as they may involve additional overhead due to their higher-level abstraction. Additionally, monitors may not be suitable for all types of synchronization problems, and in some cases, lower-level primitives may be required for optimal performance.

Producer-Consumer Problem

The producer-consumer problem involves two types of processes: producers that generate data and consumers that process data. The Producer-Consumer Problem is like a restaurant where the chef (producer) makes food and the customer (consumer) eats it. The counter (buffer: A fixed-size queue where producers place items and consumers remove items.) holds food temporarily. A special lock (semaphore) ensures the chef doesn't overflow the counter and the customer doesn't take food when it's not available. In This way, everything runs smoothly and efficiently and gives faster results.

Semaphores Used

- empty: Counts the number of empty slots in the buffer
- full: Counts the number of full slots in the buffer
- mutex: Locks access to the buffer (mutual exclusion)

Dining Philosophers Problem

The Dining Philosopher Problem states that K philosophers are seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.

There are three states of the philosopher: THINKING, HUNGRY, and EATING. Here there are two semaphores: Mutex and a semaphore array for the philosophers. Mutex is used such that no two philosophers may access the pickup or put it down at the same time. The array is used to control the behavior of each philosopher. But, semaphores can result in deadlock due to programming errors.

The Dining Philosopher Problem is a classic synchronization problem in computer science that involves multiple processes (philosophers) sharing a limited set of resources (forks) in order to perform a task (eating). In order to avoid deadlock or starvation, a solution must be implemented that ensures that each philosopher can access the resources they need to perform their task without interference from other philosophers.

One common solution to the Dining Philosopher Problem using semaphores, a synchronization mechanism that can be used to control access to shared resources. In this solution, each fork is represented by a semaphore, and a philosopher must acquire both the semaphore for the fork to their left and the semaphore for the fork to their right before they can begin eating. If a philosopher cannot acquire both semaphores, they must wait until they become available.

Deadlock

A deadlock is a situation where a set of processes is blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Necessary Conditions for Deadlock in OS

Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)

- Mutual Exclusion: Two or more resources are non-shareable (Only one process can use at a time).
- Hold and Wait: A process is holding at least one resource and waiting for resources.
- No Preemption: A resource cannot be taken from a process unless the process releases the resource.
- Circular Wait: A set of processes waiting for each other in circular form.

Deadlock detection is a process in computing where the system checks if there are any sets of processes that are stuck waiting for each other indefinitely, preventing them from moving forward. In simple words, deadlock detection is the process of finding out whether any process is stuck in a loop or not.

There are three ways to handle deadlock

- Deadlock Prevention or Avoidance
- Deadlock Recovery
- Deadlock Ignorance

Deadlock Prevention:

- Objective: To ensure that the system is designed in such a way that deadlocks are impossible.
- Method: This involves designing the system to prevent any of the four Coffman conditions (mutual exclusion, hold and wait, no preemption, and circular wait) from occurring.
- Techniques:
 - Mutual Exclusion: Design resources to be shareable where possible.
 - Hold and Wait: Ensure that processes request all resources at once or release held resources before requesting new ones.
 - No Preemption: If a process is holding resources and requests additional ones, preempt resources from other processes if necessary.
 - Circular Wait: Impose a total ordering on resources and ensure that processes request resources in a predefined order.

Deadlock Avoidance:

- Objective: To dynamically ensure that the system remains in a safe state where deadlock is avoided.
- Method: Requires knowledge of future resource requests and uses algorithms to ensure that the system always remains in a safe state.
- Techniques:
 - Banker's Algorithm: Determines whether resource allocation would leave the system in a safe state by checking against the maximum possible demands of all processes.
 - Resource Allocation Graph: Monitors the allocation of resources and requests to ensure that the system does not enter an unsafe state.

Deadlock Recovery

- Objective: To handle deadlocks after they have occurred, ensuring that the system can recover and continue operating.
- Method: Involves detecting deadlocks and taking action to resolve them.
- Techniques:
 - Process Termination: Abort one or more of the processes involved in the deadlock to break the cycle.
 - Resource Preemption: Temporarily take resources away from some processes and allocate them to others to break the deadlock.
 - Rollback: Rollback processes to a safe state and restart them to avoid deadlock.
 - Checkpointing: Save the state of processes periodically so that they can be rolled back to a recent checkpoint if needed.

Deadlock Ignorance

- Objective: To ignore the issue of deadlocks and rely on other strategies or the assumption that deadlocks are rare.
- Method: The system does not actively try to prevent, avoid, or recover from deadlocks.
- Techniques:
 - Application-Level Management: Let application developers handle deadlock issues at the application level.
 - System Simplicity: Design the system with simplicity in mind, accepting that occasional deadlocks may occur and can be manually managed or mitigated.